

开发指南

官方 Python SDK

 Copy page

Z.ai Python SDK 是一个智谱AI 官方提供的功能强大、易于使用的 Python 开发工具包，专为与智谱AI 的各种人工智能模型进行交互而设计，为 Python 开发者提供便捷、高效的AI 模型集成方案。

 最新 Python SDK 版本为 `0.2.2`，请及时更新以获取最新功能和修复。

核心优势



简单易用

Pythonic 的 API 设计，完善的文档和示例，让您快速上手



功能完整

支持智谱AI 全系列模型，包括语言、视觉、图像生成等



高性能

异步支持、连接池管理，优化的网络请求处理



类型安全

完整的类型提示，IDE 友好，减少开发错误

支持的功能

- **对话聊天**：支持单轮和多轮对话，流式和非流式响应
- **函数调用**：让 AI 模型调用您的自定义函数

- 视频生成：文本到视频的创意内容生成
- 语音处理：语音转文字、文字转语音
- 文本嵌入：文本向量化，支持语义搜索
- 智能助手：构建专业的 AI 助手应用
- 内容审核：文本和图像内容安全检测

技术规格

环境要求

- Python 版本：Python 3.8 或更高版本
- 包管理器：pip 或 poetry
- 网络要求：支持 HTTPS 连接
- API 密钥：需要有效的智谱AI API 密钥

依赖管理

SDK 采用模块化设计，您可以根据需要选择性安装功能模块：

- 核心模块：基础 API 调用功能
- 异步模块：异步和并发处理支持
- 工具模块：实用工具和辅助功能

快速开始

环境要求



Python 版本



包管理器

⚠️ 支持 Python 3.8, 3.9, 3.10, 3.11, 3.12 版本, 跨平台兼容 Windows、macOS、Linux

安装 SDK

使用 pip 安装

```
# 安装最新版本
pip install zai-sdk

# 或指定版本
pip install zai-sdk==0.2.2
```

验证安装

```
import zai
print(zai.__version__)
```

获取 API Key

1. 访问 [Z 智谱开放平台](#)
2. 注册并登录您的账户
3. 在 [API Keys](#) 管理页面创建 API Key
4. 复制您的 API Key 以供使用

💡 建议将 API Key 设置为环境变量: `export ZAI_API_KEY=your-api-key` 替代硬编码到代码中, 以提高安全性。

💡 国内智谱AI 平台使用 ZhipuAiClient 客户端



创建客户端 [>]

环境变量 直接设置

```
from zai import ZhipuAiClient
import os

# 从环境变量读取 API Key
client = ZhipuAiClient(api_key=os.getenv("ZAI_API_KEY"))

# 或者直接使用 (如果已设置环境变量)
client = ZhipuAiClient()
```

基础对话

```
from zai import ZhipuAiClient

# Initialize client
client = ZhipuAiClient(api_key="your-api-key")

# Create chat completion
response = client.chat.completions.create(
    model="glm-5.1",
    messages=[
        {"role": "user", "content": "你好, 请介绍一下自己, Z.ai!"}
    ]
)
print(response.choices[0].message.content)
```

流式对话



```
from zai import ZhipuAiClient
```

```
# Initialize client
client = ZhipuAiClient(api_key="your-api-key")

# Create chat completion
response = client.chat.completions.create(
    model='glm-4.7',
    messages=[
        {'role': 'system', 'content': '你是一个 AI 作家.'},
        {'role': 'user', 'content': '讲一个关于 AI 的故事.'},
    ],
    stream=True,
)

for chunk in response:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end='')
```

多轮对话

```
from zai import ZhipuAiClient
client = ZhipuAiClient(api_key="your-api-key")
response = client.chat.completions.create(
    model="glm-5.1", # 请填写您要调用的模型名称
    messages=[
        {"role": "user", "content": "作为一名营销专家，请为我的产品创作一个吸引人的口号"},
        {"role": "assistant", "content": "当然，要创作一个吸引人的口号，请告诉我一些关于您"},
        {"role": "user", "content": "智谱AI 开放平台"},
        {"role": "assistant", "content": "点燃未来，智谱AI 绘制无限，让创新触手可及！"},
        {"role": "user", "content": "创作一个更精准且吸引人的口号"}
    ],
)
print(response.choices[0].message.content)
```

>



```
def main():  
    # 初始化客户端  
    client = ZhipuAiClient(api_key=os.getenv("ZAI_API_KEY"))  
  
    print("欢迎使用 Z.ai 聊天机器人! 输入 'quit' 退出。")  
  
    # 对话历史  
    conversation = [  
        {"role": "system", "content": "你是一个友好的 AI 助手"}  
    ]  
  
    while True:  
        # 获取用户输入  
        user_input = input("您: ")  
  
        if user_input.lower() == 'quit':  
            break  
  
        try:  
            # 添加用户消息  
            conversation.append({"role": "user", "content": user_input})  
  
            # 创建聊天请求  
            response = client.chat.completions.create(  
                model="glm-5.1",  
                messages=conversation,  
                temperature=0.7,  
                max_tokens=1000  
            )  
  
            # 获取 AI 回复  
            ai_response = response.choices[0].message.content  
            print(f"AI: {ai_response}")  
  
            # 添加 AI 回复到对话历史  
            conversation.append({"role": "assistant", "content": ai_response})
```

```
print("再见! ")
```

```
if __name__ == "__main__":
    main()
```

错误处理

```
from zai import ZhipuAiClient
import zai

def robust_chat(message):
    client = ZhipuAiClient(api_key="your-api-key")

    try:
        response = client.chat.completions.create(
            model="glm-5.1",
            messages=[{"role": "user", "content": message}]
        )
        return response.choices[0].message.content

    except zai.core.APIStatusError as err:
        return f"API 状态错误: {err}"
    except zai.core.APITimeoutError as err:
        return f"请求超时: {err}"
    except Exception as err:
        return f"其他错误: {err}"

# 使用示例
result = robust_chat("你好")
print(result)
```

高级配置

```
import httpx
from zai import ZhipuAiClient
```

```
# 自定义 HTTP 客户端
httpx_client = httpx.Client(
    limits=httpx.Limits(
        max_keepalive_connections=20,
        max_connections=100
    ),
    timeout=30.0
)

# 创建带自定义配置的客户端
client = ZhipuAiClient(
    api_key="your-api-key",
    base_url="https://open.bigmodel.cn/api/paas/v4/",
    timeout=httpx.Timeout(timeout=300.0, connect=8.0),
    max_retries=3,
    http_client=httpx_client
)
```

高级功能

推理 (thinking)

在思考模式下，GLM-4.7 可以解决复杂的推理问题，包括数学、科学和逻辑问题。



from zhipuai import ZhipuAiClient



```
client = ZhipuAiClient(api_key='your-api-key')
```

```
response = client.chat.completions.create(
```

```
    model='glm-4.7',
```

```
    messages=[
```

```
        {"role": "system", "content": "you are a helpful assistant"},
```

```
        {"role": "user", "content": "what is the revolution of llm?"}
```

```
    ],
```

```
    stream=True,
```

```
    thinking={
```

```
        "type": "enabled"
```

```
    }
```

```
)
```

```
for chunk in response:
```

```
    if chunk.choices[0].delta.reasoning_content:
```

```
        print(chunk.choices[0].delta.reasoning_content, end='')
```

```
    if chunk.choices[0].delta.content:
```

```
        print(chunk.choices[0].delta.content, end='')
```

函数调用 (Function Calling)

函数调用允许 AI 模型调用您定义的函数来获取实时信息或执行特定操作。

定义和使用函数



定义函数 >

```
def get_weather(location, date=None):
```

```
    """获取天气信息"""
```

```
    # 模拟天气 API 调用
```

```
    return {
```

```
        "location": location,
```

```
        "date": date or "今天",
```

```
        "weather": "晴天",
```

```
        "temperature": "25°C",
```

```
        "humidity": "60%"
```

```
    }
```

```
def get_stock_price(symbol):
```

```
    """获取股票价格"""
```

```
    # 模拟股票 API 调用
```

```
    return {
```

```
        "symbol": symbol,
```

```
        "price": 150.25,
```

```
        "change": "+2.5%"
```

```
    }
```

函数描述

```
tools = [
```

```
    {
```

```
        "type": "function",
```

```
        "function": {
```

```
            "name": "get_weather",
```

```
            "description": "获取指定地点的天气信息",
```

```
            "parameters": {
```

```
                "type": "object",
```

```
                "properties": {
```

```
                    "location": {
```

```
                        "type": "string",
```

```
                        "description": "地点名称"
```

```
                    },
```

```
                    "date": {
```

```
                        "type": "string",
```

```

    },
    {
      "type": "function",
      "function": {
        "name": "get_stock_price",
        "description": "获取股票当前价格",
        "parameters": {
          "type": "object",
          "properties": {
            "symbol": {
              "type": "string",
              "description": "股票代码"
            }
          },
          "required": ["symbol"]
        }
      }
    }
  ],

  # 使用函数调用
  client = ZhipuAiClient(api_key="your-api-key")

  response = client.chat.completions.create(
    model='glm-4.7',
    messages=[
      {'role': 'user', 'content': '北京今天天气怎么样? '}
    ],
    tools=tools,
    tool_choice="auto"
  )

  # 处理函数调用
  if response.choices[0].message.tool_calls:

```



```
for tool_call in response.choices[0].message.tool_calls:
    function_name = tool_call.function.name
    function_args = json.loads(tool_call.function.arguments)

    >
    if function_name == "get_weather":
        result = get_weather(**function_args)
        print(f"天气信息: {result}")
    elif function_name == "get_stock_price":
        result = get_stock_price(**function_args)
        print(f"股票信息: {result}")
else:
    print(response.choices[0].message.content)
```

网络搜索工具

```
# 初始化客户端
client = ZhipuAiClient(api_key="your-api-key")

# 使用网络搜索工具
response = client.chat.completions.create(
    model='glm-4.7',
    messages=[
        {'role': 'system', 'content': 'You are a helpful assistant.'},
        {'role': 'user', 'content': 'What is artificial intelligence?'},
    ],
    tools=[
        {
            'type': 'web_search',
            'web_search': {
                'search_query': 'What is artificial intelligence?',
                'search_result': True,
            },
        },
    ],
    temperature=0.5,
    max_tokens=2000,
)

print(response)
```

多模态处理

图像理解

```
from zai import ZhipuAiClient
```

```
def encode_image(image_path):  
    """将图像编码为 base64 格式"""  
    with open(image_path, 'rb') as image_file:  
        return base64.b64encode(image_file.read()).decode('utf-8')
```

```
client = ZhipuAiClient(api_key="your-api-key")
```

```
# 方式1: 使用图像URL
```

```
response = client.chat.completions.create(  
    model="glm-4.6v",  
    messages=[  
        {  
            "role": "user",  
            "content": [  
                {  
                    "type": "text",  
                    "text": "这张图片里有什么？请详细描述。"  
                },  
                {  
                    "type": "image_url",  
                    "image_url": {  
                        "url": "https://example.com/image.jpg"  
                    }  
                }  
            ]  
        }  
    ]  
)
```

```
print(response.choices[0].message.content)
```

```
# 方式2: 使用base64编码的图像
```

```
base64_image = encode_image('path/to/your/image.jpg')
```

```
response = client.chat.completions.create(  
    model="glm-4.6v",  
    messages=[
```

 BigModel



```
{
  "role": "user",
  "content": [
    {
      "type": "text",
      "text": "分析这张图片中的内容"
    },
    {
      "type": "image_url",
      "image_url": {
        "url": f"data:image/jpeg;base64,{base64_image}"
      }
    }
  ]
}
```

```
)

print(response.choices[0].message.content)
```

图像生成



```
# Initialize client
client = ZhipuAiClient(api_key="your-api-key")

# 图像生成
response = client.images.generations(
    model="cogview-3",
    prompt="一幅美丽的山水画，中国传统风格，水墨画",
    size="1024x1024",
    quality="standard",
)

image_url = response.data[0].url
print(f"生成的图像URL: {image_url}")

# 高质量图像生成
response = client.images.generations(
    model="cogview-3-plus",
    prompt="未来城市的概念设计，科幻风格，高清细节",
    size="1024x1024",
    quality="hd",
)

image_url = response.data[0].url
print(f"生成的图像URL: {image_url}")
```

视频生成



from zhipuai import ZhipuAiClient
import time



```
client = ZhipuAiClient(api_key="your-api-key")

# 提交生成任务
response = client.videos.generations(
    model="cogvideox-3", # 使用的视频生成模型
    image_url=image_url, # 提供的图片 URL 地址或者 Base64 编码
    prompt="让画面动起来",
    quality="speed", # 输出模式, "quality"为质量优先, "speed"为速度优先
    with_audio=True,
    size="1920x1080", # 视频分辨率, 支持最高 4K (如: "3840x2160")
    fps=30, # 帧率, 可选为 30 或 60
)
print(response)

# 获取生成结果
time.sleep(60) # 等待一段时间以确保视频生成完成
result = client.videos.retrieve_videos_result(id=response.id)
print(result)
```

文本嵌入



```
from zai import ZhipuAiClient

client = ZhipuAiClient(api_key="your-api-key")

>

response = client.embeddings.create(
    model="embedding-3",
    input=[
        "这是第一段文本",
        "这是第二段文本",
        "这是第三段文本"
    ]
)

for i, embedding in enumerate(response.data):
    print(f"文本{i+1}的嵌入向量维度: {len(embedding.embedding)}")
    print(f"前5个维度的值: {embedding.embedding[:5]}")

# 计算文本相似度
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def calculate_similarity(texts):
    """计算文本间的相似度"""
    response = client.embeddings.create(
        model="embedding-2",
        input=texts
    )

    embeddings = [data.embedding for data in response.data]
    embeddings_array = np.array(embeddings)

    # 计算余弦相似度
    similarity_matrix = cosine_similarity(embeddings_array)

    return similarity_matrix

# 使用示例
texts = [
    "我喜欢吃苹果",
    "苹果是我最爱的水果",
```

```
similarity = calculate_similarity(texts)
print("相似度矩阵:")
print(similarity)
```

流式处理



```
class StreamProcessor:
    def __init__(self, client):
        self.client = client
        self.full_response = ""

    def stream_chat(self, messages, model="glm-5.1", callback=None):
        """流式聊天处理"""
        stream = self.client.chat.completions.create(
            model=model,
            messages=messages,
            stream=True
        )

        self.full_response = ""
        for chunk in stream:
            if chunk.choices[0].delta.content is not None:
                content = chunk.choices[0].delta.content
                self.full_response += content

                if callback:
                    callback(content, self.full_response)
                else:
                    print(content, end="", flush=True)

        print() # 换行
        return self.full_response

# 使用示例
processor = StreamProcessor(client)

# 自定义回调函数
def on_token_received(token, full_text):
    # 可以在这里实现实时处理逻辑
    print(token, end="", flush=True)

response = processor.stream_chat(
    messages=[{"role": "user", "content": "写一个 Python 函数来计算斐波那契数列"}],
    callback=on_token_received
)
```



GitHub 仓库

查看源代码、提交问题、参与贡献



API 参考

查看完整的 API 文档



示例项目

浏览更多实际应用示例



最佳实践

学习 SDK 使用的最佳实践

ⓘ 本 SDK 基于智谱AI 最新的 API 规范开发，确保与平台功能保持同步更新。建议定期更新到最新版本以获得最佳体验。